Lab 5 (Arithmetic Operations in Prolog)

Objective: Understand how Prolog performs arithmetic operations, how numbers are evaluated, and how arithmetic expressions differ from unification.

Numeric Types in Prolog:
Prolog supports:

- Integers: e.g., 5, -12
- Floats: e.g., 3.14, -0.5

Important Arithmetic Operators

Operator	Meaning	Example
+	Addition	2 + 3
-	Subtraction	7 - 4
*	Multiplication	3 * 2
/	Float Division	7 / 2 → 3.5
//	Integer Division	7 // 2 → 3
mod	Modulus (remainder)	7 mod 2 → 1
^	Power	2 ^ 3 → 8

The is/2 Operator: Unlike unification (=), the is operator evaluates an arithmetic expression on the right-hand side.

- ✓ is/2 Means: the is predicate that takes 2 arguments.

Example:

```
?- X is 3 + 5.
X = 8.
```

`is` evaluates `3 + 5` and binds `X` to `8`.

Comparison: `=` vs `is`

```
?- X = 3 + 5.
X = 3+5.          % Treated as a structure, not evaluated

?- X is 3 + 5.
X = 8.            % Correct: evaluated
```

Arithmetic Comparisons

Operator	Meaning
=:=	Equal after evaluation
=\=	Not equal
<	Less than
>	Greater than
=<	Less than or equal
>=	Greater than or equal

Example:

```
?- 5 =:= 2 + 3.
true.

?- X is 2 + 3, X > 4.
X = 5.
```

Common Pitfalls

Mistake	Problem	Fix
X = 3 + 4	No evaluation	Use X is 3 + 4
3 + 4 is X	Invalid (wrong direction)	RHS must be expression
X is Y + 1 when Y is unbound	Error: instantiation fault	Ensure Y is bound first

Error: Instantiation Fault
You'll see this error when you try to evaluate an arithmetic expression where a variable is unbound.

Faulty Example:

```
?- X is Y + 1.
```

- ✓ You're asking Prolog to evaluate Y + 1
- ✓ But Y is not yet assigned a value (it's unbound)
- ✓ Prolog cannot compute an expression with an unknown value
- ✓ So it throws:

ERROR: Arguments are not sufficiently instantiated

This is called an instantiation fault.

Correct Example:

First, bind Y to a value before using `is`:

```
?- Y = 4, X is Y + 1.
Y = 4,
X = 5.
```

Now:

`Y` is bound to 4
`X is Y + 1` becomes `X is 4 + 1`
Result: `X = 5`

Program: Calculate Area and Perimeter of a Rectangle

```
% area(Length, Width, Area)
area(L, W, A) :-
    A is L * W.

% perimeter(Length, Width, Perimeter)
perimeter(L, W, P) :-
        P is 2 * (L + W).
```

Example Queries:

```
?- area(5, 3, A).
A = 15.

?- perimeter(5, 3, P).
P = 16.
```

Input validation using basic guards (conditions like R > 0).

Prolog Program: Geometry Calculator

```
% Rectangle: Area = L * W
area_rectangle(L, W, A) :-
    L > 0, W > 0,                    % Input validation
    A is L * W.

% Rectangle: Perimeter = 2 * (L + W)
perimeter_rectangle(L, W, P) :-
    L > 0, W > 0,
    P is 2 * (L + W).

% Circle: Area = pi * R^2
area_circle(R, A) :-
    R > 0,
    A is 3.14159 * R * R.

% Circle: Circumference = 2 * pi * R
circumference_circle(R, C) :-
        R > 0,
        C is 2 * 3.14159 * R.
```

Sample Queries:

```
?- area_rectangle(5, 4, A).
A = 20.

?- perimeter_rectangle(5, 4, P).
P = 18.
```

```
?- area_circle(2, A).
A = 12.56636.

?- circumference_circle(2, C).
C = 12.56636.
```

Invalid Inputs:

```
?- area_circle(-2, A).
false.     % Rejected due to R > 0
```

Example Program: Student Gradebook Analyzer

This program will:

- ✓ Store student marks
- ✓ Calculate total and average
- ✓ Determine grade
- ✓ Display full report per student

```
% --- Student database ---
student(john, [85, 92, 78]).
student(mary, [60, 70, 65]).
student(ali, [90, 88, 95]).
student(sara, [40, 38, 45]).

% --- Sum of a list ---
sum_list([], 0).
sum_list([H | T], Sum) :-
    sum_list(T, Rest),
    Sum is H + Rest.

% --- Average ---
average(List, Avg) :-
    sum_list(List, Sum),
    length(List, Count),
    Count > 0,
    Avg is Sum / Count.

% --- Grade logic ---
grade(Avg, 'A') :- Avg >= 85, !.
grade(Avg, 'B') :- Avg >= 70, !.
grade(Avg, 'C') :- Avg >= 50, !.
grade(_, 'F').

% --- Full report for a student ---
report(Name) :-
    student(Name, Marks),
    sum_list(Marks, Total),
    average(Marks, Avg),
    grade(Avg, Grade),
    format("Student: ~w~nMarks: ~w~nTotal: ~w~nAverage:     ~2f~nGrade:  ~w~n~n",  [Name,  Marks,
Total, Avg, Grade]).
```

Sample Query:

```
?- report(mary).
```

Output:

```
Student: mary
Marks: [60,70,65]
Total: 195
Average: 65.00
Grade: C
```

Step-by-step execution of the query:

```
?- report(mary).
```

Facts:

```
student(mary, [60, 70, 65]).
```

1. Match report(mary):

```
report(Name) :-
    student(Name, Marks),
    sum_list(Marks, Total),
    average(Marks, Avg),
    grade(Avg, Grade),
    format(...).
```

Here, Name = mary

2. Match student(mary, Marks):

From the fact:

```
student(mary, [60, 70, 65]).
```

So:

Marks = [60, 70, 65]

3. Call sum_list([60, 70, 65], Total)

This uses recursion:

```
sum_list([60,70,65], T1)
→ sum_list([70,65], T2)
→ sum_list([65], T3)
→ sum_list([], 0) ← base case
```

Now unwind:

- T3 = 0 → T2 = 0 + 65 = 65
- T1 = 65 + 70 = 135
- Total = 135 + 60 = 195

So:

Total = 195

4. Call average([60, 70, 65], Avg)

- Already know: Sum = 195
- length([60,70,65], Count) → Count = 3
- Avg is 195 / 3 → Avg = 65.0

So:

Avg = 65.0

5. Call grade(65.0, Grade)

Grade rules:

```
grade(Avg, 'A') :- Avg >= 85, !.       % fails
grade(Avg, 'B') :- Avg >= 70, !.       % fails
grade(Avg, 'C') :- Avg >= 50, !.       % succeeds
grade(_, 'F').                         % skipped
```

```
→ Grade = 'C'
```

6. Final Step: Call format/2

```
format( "Student: ~w~nMarks: ~w~nTotal: ~w~nAverage: ~2f~nGrade: ~w~n~n",[mary, [60, 70, 65],
195, 65.0, 'C']).
```

Output:

```
Student: mary
Marks: [60,70,65]
Total: 195
Average: 65.00
Grade: C
```

Summary of Formatting Codes: Code	Meaning
~w	Write Prolog term (any value)
~n	Newline
~d	Decimal integer
~f	Floating point number
~2f	Float with 2 decimal places
~a	Print atom/text without quotes

Assignment Title: Salary Calculator with Tax Bracket Logic in Prolog

Objective:

Develop a Prolog program that:

- Calculates gross salary, tax, and net salary
- Applies different tax rates based on salary brackets
- Uses arithmetic operations, conditionals, and formatted output

Requirements:

1. Store employee data in this format:

```
employee(Name, BasicSalary, Allowance).
```

2. Calculate Gross Salary:

```
GrossSalary is BasicSalary + Allowance.
```

3. Apply Tax Rules:

Gross Salary	Tax Rate
< 30,000	0%
30,000–60,000	10%
60,001–100,000	20%
Above 100,000	30%

4. Calculate:

```
Tax = Gross * Rate
Net Salary = Gross – Tax
```

5. Print formatted report:

```
Employee: Alice
Basic Salary: 40000
Allowance: 10000
Gross Salary: 50000
Tax: 5000
Net Salary: 45000
```

6. Use:

```
is/2, +, *, -
Conditional clauses with cut (!) for tax brackets
format/2 for output
```